Why optimisation does not compose

Simon Frost

Table of contents

1 Overview 1

2 Setup 2

3 The counterexample 2

4 Optimising each component in isolation 3

5 Why it fails 4

6 The contextual repair 5

7 Pointwise maximisation at the boundary is not a strategy 6

8 Summary 7

9 References 8

1 Overview

For a fixed strategy, the semantics of open influence diagrams is compositional: the reward kernel of a composite is the composite of the reward kernels (the first vignette, Open influence diagrams and composition). It is tempting to conclude that optimisation is compositional too: optimise each component on its own, then compose the optimal strategies. It is not. A study of serial influence semantics, carried out in three proof assistants before this package was written, refutes it with counterexamples (its claim `INF-ISOLATED-OBSTRUCTION`). This vignette builds one of them as two open influence diagrams, shows why it fails, repairs it with contextual optimisation, and shows why the obvious alternative repair, maximising separately at each boundary state, is not a strategy at all (the study's `INF-HIDDEN-OBSTRUCTION`).

The phenomenon is familiar from limited-memory influence diagrams, where improving one policy at a time against the others reaches a local optimum that need not be global (Lauritzen

and Nilsson 2001). What the open setting adds is a precise account of the context a component must be optimised against, and of what crosses the boundary.

2 Setup

```
using CategoricalInfluenceDiagrams
```

3 The counterexample

There are two components and three binary variables, with labels `no` and `yes`.

- The early component generates a fair hidden bit `H` and holds an action `A` that observes nothing. Taking the action (`A = yes`) costs 1.
- The late component holds an action `B` that observes `A` only. It earns 4 when the early action was taken, and 2 when `B` matches the hidden bit.

The boundary carries `H` hidden and `A` observed: the late decision may read what the early decision did, but not the bit.

```
bool = [:no, :yes]
early = Open(influence_diagram(:H ⇒ bool, :A ⇒ bool;
                               mechanisms = [:H ⇒ ()],
                               decisions = [:A ⇒ Symbol[]],
                               utilities = [:EarlyCost ⇒ :A], closed = false);
             outputs = [:H, :A], observed = [:A])
late = Open(influence_diagram(:H ⇒ bool, :A ⇒ bool, :B ⇒ bool;
                              decisions = [:B ⇒ :A],
                              utilities = [:LateBenefit ⇒ :A, :HiddenMatch ⇒ (:H,
:B)],
                              closed = false);
            inputs = [:H, :A], observed = [:A])
println(early)
println(late)
```

```
OpenInfluenceDiagram(2 variables, 1 mechanisms, 1 decisions, 1 utilities; Symbol[]
-> [:H, :A!])
OpenInfluenceDiagram(3 variables, 0 mechanisms, 1 decisions, 2 utilities; [:H, :A!]
-> Symbol[])
```

```
ax(x) = FiniteAxis(x, bool)
kernels = Dict{Symbol,FiniteKernel}(:H ⇒ state(FiniteSpace(:H, bool), [0.5, 0.5]))
utilities = Dict{Symbol,AbstractUtility}(:EarlyCost ⇒ TabularUtility([ax(:A)], [0,
-1]),
                                         :LateBenefit ⇒ TabularUtility([ax(:A)],
                                         [0, 4]),
```

```
                                        :HiddenMatch ⇒ TabularUtility([ax(:H),
                                        ax(:B)],
                                                                      [2 0; 0 2]))
nothing # hide
```

The composite is closed. Whatever the late policy, it cannot know the bit, so it matches it half the time and earns 1 from `HiddenMatch` on average. The early action therefore decides everything: not taking it is worth 1, taking it is worth `-1 + 4 + 1 = 4`. The table lists every strategy of the composite: the early action, and the late action for each value of `A`.

```
whole = compose(early, late)
constant(d, a) = Strategy(d ⇒ ConstantPolicy(a))
late_policies = all_deterministic_policies(:B, [ax(:A)], ax(:B))
[(A = a, B = Tuple(policy_table(p)),
  value = contextual_value(whole, kernels, utilities,
                           merge_strategies(constant(:A, a), Strategy(:B ⇒ p))))
 for a in bool for p in late_policies]
```

```
8-element Vector{@NamedTuple{A::Symbol, B::Tuple{Symbol, Symbol}, value::Float64}}:
 (A = :no, B = (:no, :no), value = 1.0)
 (A = :no, B = (:yes, :no), value = 1.0)
 (A = :no, B = (:no, :yes), value = 1.0)
 (A = :no, B = (:yes, :yes), value = 1.0)
 (A = :yes, B = (:no, :no), value = 4.0)
 (A = :yes, B = (:yes, :no), value = 4.0)
 (A = :yes, B = (:no, :yes), value = 4.0)
 (A = :yes, B = (:yes, :yes), value = 4.0)
```

`optimize` on the closed composite finds the optimum, 4, with the early action taken:

```
best = optimize(whole, kernels, utilities)
best.expected_utility, only(policy_table(best.strategy[:A]))
```

```
(4.0, :yes)
```

4 Optimising each component in isolation

Optimising a component "in isolation" means optimising it against the context it actually receives, with nothing downstream: its true prior, and a zero continuation. The early component has no inputs, and its outputs `(H, A)` have four joint states. Against a zero continuation it sees only the cost of the action, so it does not take it:

```
iso_early = contextual_optimize(early, kernels, utilities; continuation = zeros(4))
iso_early.expected_utility, only(policy_table(iso_early.strategy[:A]))
```

```
(0.0, :no)
```

The late component, given the boundary law that this choice induces on `(H, A)` (`A = no` for certain, `H` fair), finds every legal policy worth the same 1, and the search returns the first:

```
ν = boundary_law(early, kernels, utilities, iso_early.strategy)
iso_late = contextual_optimize(late, kernels, utilities; prior = ν)
(boundary_law = ν,
 values = [Tuple(policy_table(p)) ⇒
           contextual_value(late, kernels, utilities, Strategy(:B ⇒ p); prior = ν)
           for p in late_policies],
 chosen = Tuple(policy_table(iso_late.strategy[:B])))
```

```
(boundary_law = [0.5, 0.5, 0.0, 0.0], values = [(:no, :no) => 1.0, (:yes, :no) =>
1.0, (:no, :yes) => 1.0, (:yes, :yes) => 1.0], chosen = (:no, :no))
```

Each component is optimal in its own context. Composed, they are worth 1, where the composite attains 4:

```
isolated = contextual_value(whole, kernels, utilities,
                            merge_strategies(iso_early.strategy, iso_late.strategy))
isolated, best.expected_utility, isolated < best.expected_utility
```

```
(1.0, 4.0, true)
```

5 Why it fails

Two things go wrong, and both are about the boundary.

The prefix's choice changes the suffix's value. The late component's value from a boundary state is not fixed: it is 4 more when the early action was taken. That value is the continuation of the early component, its value function over the boundary states. For the late policy that ignores `A` and plays `no`, `continuation_value` gives it over the joint states of `(H, A)`, `H` varying fastest:

```
τ = Strategy(:B ⇒ first(late_policies))
c = continuation_value(late, kernels, utilities, τ)
[s ⇒ v for (s, v) in zip(joint_states(dom(interpret(late, kernels, utilities, τ))),
c)]
```

```
4-element Vector{Pair{Tuple{Symbol, Symbol}, Float64}}:
   (:no, :no) => 2.0
  (:yes, :no) => 0.0
  (:no, :yes) => 6.0
 (:yes, :yes) => 4.0
```

The early component optimised against a zero continuation never sees the 4: it compares a cost of 1 against nothing. Isolation does not merely lose some precision; it optimises a different objective.

The hidden boundary state cannot be read. The late component would like to match `H`, which would earn 2 instead of 1. But `H` is hidden at the boundary, so a legal late policy is a function of `A` alone. Against the law of the isolated early choice no legal policy does better than 1, and nothing the late component can do signals to the early component that the action is worth taking.

6 The contextual repair

The value of a composite is the value of either component in the context the other induces. For a prefix `f` with strategy σ, a suffix `g` with strategy τ, a prior μ and a continuation `c`, `EU_{f;g}(σ, τ; μ, c) = EU_f(σ; μ, c_τ) = L_σ + EU_g(τ; ν_σ, c)`, where `c_τ` is the suffix's value function, `ν_σ` the boundary law the prefix induces and `L_σ` the reward the prefix earns on its own (the study's `INF-FIXED-CONTEXT`). Here μ and `c` are trivial, because the composite is closed.

So optimise the early component against the late component's value function instead of against zero. It now takes the action, and attains the optimum:

```
repaired = contextual_optimize(early, kernels, utilities; continuation = c)
repaired.expected_utility, only(policy_table(repaired.strategy[:A]))
```

```
(4.0, :yes)
```

The continuation depends on the suffix's strategy, so the full search runs over the suffix's strategies, optimising the prefix against each value function and keeping the best:

```
suffix_first = maximum(contextual_optimize(early, kernels, utilities;
                                           continuation = continuation_value(late,
kernels,

utilities,

                                                                                 Strategy
                                                                                 (:B ⇒
                                                                                 p))).expected_utility
                       for p in late_policies)
```

```
4.0
```

The other form of the identity runs the other way: for each strategy of the prefix, add its own reward to the optimum of the suffix against the boundary law it induces. That reaches the optimum too:

```
prefix_first = maximum(begin
                           σ = constant(:A, a)
                           L = contextual_value(early, kernels, utilities, σ;
                                                continuation = zeros(4))
                           ν_σ = boundary_law(early, kernels, utilities, σ)
                           L + contextual_optimize(late, kernels, utilities;
```

```
                                                    prior = ν_σ).expected_utility
                        end
                        for a in bool)
```

```
4.0
```

Neither form optimises a component on its own. Each optimises one component against a context that the other component's strategy determines, and searches over that strategy. This is why the package provides `contextual_value`, `contextual_optimize`, `continuation_value` and `boundary_law`, and no operation that optimises the components of a composite one by one and composes the results.

7 Pointwise maximisation at the boundary is not a strategy

A different repair suggests itself for the late component: from each boundary state, pick the action with the higher value. Against the boundary law of the isolated early choice that is worth 2, more than the legal optimum of 1:

```
cs = [continuation_value(late, kernels, utilities, constant(:B, b)) for b in bool]
pointwise = sum(ν .* max.(cs...))
legal = contextual_optimize(late, kernels, utilities; prior = ν)
(pointwise = pointwise, legal = legal.expected_utility)
```

```
(pointwise = 2.0, legal = 1.0)
```

But the pointwise choice is not a policy of the late decision. At the boundary states `(H = no, A = no)` and `(H = yes, A = no)`, which agree on `A`, it picks different actions:

```
states_HA = joint_states(dom(interpret(late, kernels, utilities, constant(:B,
:no))))
[s ⇒ bool[argmax([cs[1][j], cs[2][j]])] for (j, s) in enumerate(states_HA)]
```

```
4-element Vector{Pair{Tuple{Symbol, Symbol}, Symbol}}:
   (:no, :no) => :no
  (:yes, :no) => :yes
  (:no, :yes) => :no
 (:yes, :yes) => :yes
```

It reads the hidden bit. Written as a policy, the late decision would need `H` in its information set, and the typed-interface rule refuses a decision that reads a hidden input (rule 3):

```
peeking = influence_diagram(:H ⇒ bool, :A ⇒ bool, :B ⇒ bool;
                            decisions = [:B ⇒ (:H, :A)],
                            utilities = [:LateBenefit ⇒ :A, :HiddenMatch ⇒ (:H,
:B)],
                            closed = false)
```

```
try
    Open(peeking; inputs = [:H, :A], observed = [:A])
catch e
    sprint(showerror, e)
end
```

```
"DiagramInterfaceError: rule 3 (a decision reads only internal or observed
input-foot variables) is violated: decision :B has the hidden input :H in its
information set; declare it observed or remove the arc"
```

Declaring `H` observed makes the policy legal, and the late component is then worth 2 against the same law. But that changes the problem: the early component hides `H`, and the two no longer compose.

```
peek = Open(peeking; inputs = [:H, :A], observed = [:H, :A])
contextual_optimize(peek, kernels, utilities; prior = ν).expected_utility
```

```
2.0
```

```
try
    compose(early, peek)
catch e
    sprint(showerror, e)
end
```

```
"ObservabilityMismatchError: compose: outputs(A) vs inputs(B): variable :H at
position 1 is :hidden on the left but :observed on the right; glued interface
variables must agree on observability"
```

8 Summary

In the counterexample each component, optimised against the context it receives with nothing downstream, chooses well for itself, and the composite of those choices is worth 1 where the optimum is 4. The early decision cannot see the reward its action makes possible downstream, and the late decision cannot read the hidden state that would let it do better. Optimising the early component against the late component's value function, or the late component against the boundary law of each early choice, recovers the optimum; maximising pointwise over hidden boundary states does not give a policy, and the typed-interface rule refuses the information arc it would need. Optimisation of open influence diagrams is therefore always contextual: `optimize` solves a closed composite, and `contextual_optimize` solves one component against an explicit prior and continuation.

9 References

Lauritzen, Steffen L., and Dennis Nilsson. 2001. “Representing and Solving Decision Problems with Limited Information.” *Management Science* 47 (9): 1235–51. https://doi.org/10.1287/mnsc.47.9.1235.9779.